

1 Main Data Stuff

1.1 data_array

Array of the visibility data, shape: (Nblts, Nfreqs, Npols), type = complex float, in units of self.vis_units.

1.2 flag_array

Boolean flag, True is flagged, same shape as data_array.

1.3 time_array

Array of times, center of integration, shape (Nblts), units Julian Date.

1.4 ant_1_array

Array of numbers for the first antenna, which is matched to that in the antenna_numbers attribute. Shape (Nblts), type = int.

1.5 ant_2_array

Array of numbers for the second antenna, which is matched to that in the antenna_numbers attribute. Shape (Nblts), type = int.

1.6 freq_array

Array of frequencies, center of the channel, shape (Nfreqs,), units Hz.

2 Super Important Variables that contribute to data

2.1 Nbls

Number of baselines.

2.2 Nblts

Number of baseline-times (i.e. number of spectra). Not necessarily equal to Nbls * Ntimes.

2.3 baseline_array

Array of baseline numbers, shape (Nblts), type = int; by default baseline = 2048 * ant1 + ant2 + 2¹⁶, though other conventions are available. This is to give each baseline a distinct code/value, I'm imagining.

2.4 Nfreqs

Number of frequency channels.

2.5 Npols

Number of polarizations.

2.6 polarization_array

Array of polarization integers, shape (Npols). AIPS Memo 117 says: pseudo-stokes 1:4 (pI, pQ, pU, pV); circular -1:-4 (RR, LL, RL, LR); linear -5:-8 (XX, YY, XY, YX). NOTE: AIPS Memo 117 actually calls the pseudo-Stokes polarizations “Stokes”, but this is inaccurate as visibilities cannot be in true Stokes polarizations for physical antennas. We adopt the term pseudo-Stokes to refer to linear combinations of instrumental visibility polarizations (e.g. pI = xx + yy). A value of 0 indicates that the polarization is different for different spectral windows and is only allowed if flex_spw_polarization_array is defined (not None).

2.7 Ntimes

Number of times.

2.8 channel_width

Width of frequency channels (Hz). Array of shape (Nfreqs), type = float.

2.9 integration_time

Length of the integration in seconds, shape (Nblts). The product of the integration_time and the nsample_array value for a visibility reflects the total amount of time that went into the visibility. Best practice is for the integration_time to reflect the length of time a visibility was integrated over (so it should vary in the case of baseline-dependent averaging and be a way to do selections for differently integrated baselines). Note that many files do not follow this convention, but it is safe to assume that the product of the integration_time and the nsample_array is the total amount of time included in a visibility.

2.10 nsample_array

Number of data points averaged into each data element. NOT required to be an integer, type = float, same shape as data_array. The product of the integration_time and the nsample_array value for a visibility reflects the total amount of time that went into the visibility. Best practice is for the nsample_array to be used to track flagging within an integration_time (leading to a decrease of the nsample array value below 1) and LST averaging (leading to an increase in the nsample array value). So datasets that have not been LST averaged should have nsample array values less than or equal to 1. Note that many files do not follow this convention, but it is safe to assume that the product of the integration_time and the nsample_array is the total amount of time included in a visibility.

3 Phase Center Stuff

3.1 Nphase

Specifies the number of phase centers contained within the data set.

3.2 phase_center_app_dec

Apparent Declination of phase center in the topocentric frame of the observatory, units radians. Shape (Nblts,), type = float.

3.3 phase_center_app_ra

Apparent right ascension of phase center in the topocentric frame of the observatory, units radians. Shape (Nblts,), type = float.

3.4 phase_center_catalog

Dictionary that acts as a catalog, containing information on individual phase centers. Keys are the catalog IDs of the different phase centers in the object (matched to the parameter phase_center_id_array). At a minimum, each dictionary must contain the keys: 'cat_name' giving the phase center name (this does not have to be unique, non-unique values can be used to indicate sets of phase centers that make up a mosaic observation), 'cat_type', which can be 'sidereal' (fixed position in RA/Dec), 'ephem' (position in RA/Dec which moves with time), 'driftscan' (fixed position in Az/El, NOT the same as the old phase_type = 'drift') or 'unprojected' (baseline coordinates in ENU, but data are not phased, similar to the old phase_type = 'drift') 'cat_lon' (longitude coord, e.g. RA, either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for ephem type phase centers), 'cat_lat' (latitude coord, e.g. Dec., either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for ephem type phase centers), 'cat_frame' (coordinate frame, e.g. icrs, must be

a frame supported by astropy). Other optional keys include ‘cat_epoch’ (epoch and equinox of the coordinate frame, not needed for frames without an epoch (e.g. ICRS) unless there is proper motion), ‘cat_times’ (times for the coordinates, only used for ‘ephem’ types), ‘cat_pm_ra’ (proper motion in RA), ‘cat_pm_dec’ (proper motion in Dec), ‘cat_dist’ (physical distance to the source in parsec, useful if parallax is important, either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for ephem type phase centers.), ‘cat_vrad’ (rest frame velocity in km/s, either a single value or a one dimensional array of length Npts –the number of ephemeris data points– for ephem type phase centers.), and ‘info_source’ (describes where catalog info came from). See the documentation of the phase method for more details.

3.5 phase_center_frame_pa

Position angle between the hour circle (which is a great circle that goes through the target position and both poles) in the apparent/topocentric frame, and the frame given in the phase_center_frame attribute. Shape (Nblts,), type = float.

3.6 phase_center_id_array

Maps individual indices along the Nblt axis to a key in phase_center_catalog, which maps to a dict containing the other metadata for each phase center. Shape (Nblts), type = int.

4 Spectral Window

4.1 Nspws

Number of spectral windows (ie non-contiguous spectral chunks). These are how we slice the frequency channels, i.e. which ones we look at. We usually don’t overlap them

4.2 spw_array

Array of spectral window numbers, shape (Nspws). Gives them a name.

4.3 flex_spw_id_array

Maps individual channels along the frequency axis to individual spectral windows, as listed in the spw_array. Shape (Nfreqs), type = int. So we can tell which channel is in which spectral window.

5 Not as Important

5.1 `lst_array`

Array of local apparent sidereal times (LAST) at the center of integration, shape (Nblts), units radians. Kind of an alternate to `time_array`. Does the same thing, but in a different measure of time.

5.2 `uvw_array`

Projected baseline vectors relative to phase center, shape (Nblts, 3), units meters. Convention is: $uvw = xyz(\text{ant2}) - xyz(\text{ant1})$. Note that this is the Miriad convention but it is different from the AIPS/FITS convention (where $uvw = xyz(\text{ant1}) - xyz(\text{ant2})$).

5.3 `telescope`

`pyuvdata.Telescope` object containing the telescope metadata.

5.4 `history`

String of history, units English.

5.5 `vis_units`

Visibility units, options are: “uncalib”, “Jy” or “K str”.

5.6 `Nants_data`

Number of antennas with data present (i.e. number of unique entries in `ant_1_array` and `ant_2_array`). May be smaller than the number of antennas in the array.